

Log Sentinel

Análise post-mortem de logs Apache

Equipe: Elder · Aryan · Rodrigo · Helena

Engenharia de Software II — IFBA 2026.1

Apresentação AV3 — 17 de junho de 2026

github.com/202316360036/log-sentinel

Visão do projeto

Problema. Investigar incidentes em logs Apache é lento, propenso a erro e exige conhecimento de regex e CLI que o sysadmin médio não tem tempo de manter.

Solução. Suíte offline (CLI + GUI) para análise post-mortem. Lê logs locais, detecta padrões, exporta relatório JSON reproduzível com hash SHA-256 do log.

Escopo MVP — fazer pouco e fazer bem:

- Três detectores: força bruta, scanner de vulnerabilidade, pico de tráfego.
- Zero requisições de rede em runtime (privacidade por design).
- Cadeia de custódia via hash embutido no relatório.

Pipeline e propriedades emergentes



Funcionais (PEF):

- PEF-01 — Detecção de força bruta a partir de log bruto.
- PEF-04 — Auditoria batch consolidada (um JSON por diretório).
- PEF-05 — Paridade CLI e GUI sobre o mesmo Core.

Não-funcionais (PENF):

- PENF-02 — Uso de RAM até 200 MB independente do tamanho do log.
- PENF-06 — GUI não congela durante processamento (QThread).
- PENF-10 — Análise totalmente offline (validado por linter).

Sete dimensões de confiança

Avizienis, Laprie, Randell e Landwehr (2004); ISO/IEC 25010

Dimensão	Mecanismo principal	Estado
Safety	Read-only sobre o log, idempotência, S5	Planejado
Security	Offline by design, B4, B5, S6, modelo de ameaças	Planejado
Reliability	Isolamento por arquivo, determinismo, cobertura alvo 80%	Parcial
Availability	App local, PyInstaller, sem deps em runtime	Ativo
Maintainability	Camadas, Strategy/Factory, ruff + pyright + Sonar	Ativo
Resilience	Parser fail-fast por linha (B2), S4, mensagens humanizadas	Planejado
Privacy	Offline, --anonymize-ips, sem telemetria	Planejado

| Defesa em profundidade — barreiras e salvaguardas

Barreiras (preventivas)

- B0** — Lint e type-check na CI — **Ativo**
- B1** — Validação de caminho de arquivo — Planejado
- B2** — Parser fail-fast por linha — Planejado
- B3** — Isolamento por arquivo no batch — Planejado
- B4** — Limite `--max-lines / --max-bytes` — Planejado
- B5** — Sanitização da saída JSON — Planejado
- B6** — GUI em QThread — Planejado
- B7** — Lint cross-layer (core → cli/gui) — Planejado
- B8** — Streaming + hash incremental — Planejado

Salvaguardas (detecção e contenção)

- S0** — Cobertura de testes — **Parcial**
- S1** — Mensagens humanizadas — Planejado
- S2** — Logging estruturado — Planejado
- S3** — Barra de progresso com aborto — Planejado
- S4** — Lista de linhas rejeitadas — Planejado
- S5** — Confirmação antes de sobrescrever — Planejado
- S6** — Hash SHA-256 do log — Planejado
- S7** — Versão do app no JSON — Planejado
- S8** — SonarCloud / SAST — **Ativo**

Hoje só B0 e S8 estão ativos; S0 parcial. Demais defesas mapeadas para sprints até AV5.

Matriz Condição Latente × Defesas

ID	Descrição resumida	Defesas
CL-01	Regex aceita silenciosamente linha mal formada	B2 · S0 · S4
CL-02	DoS por log envenenado gigante	B1 · B4 · S3
CL-03	.read() causa MemoryError em log grande	B4 · B8 · S0
CL-04	GUI congela em batch longo	B6 · S1 · S3
CL-05	Script no JSON executa em visualizador web	B5 · S0
CL-06	Acoplamento core → cli/gui (regressão)	B0 · B7 · S0
CL-09	Cron sobrescreve relatório anterior	S1 · S5 · S7

Princípio: cada condição latente é coberta por pelo menos duas defesas. Catálogo completo CL-01 a CL-10 no documento 01.

Perigos prioritários

Leveson (2011) — STAMP/STPA; IEC 61508; NIST SP 800-30

ID	Perigo	Severidade	Probabilidade	Risco
HZ-01	Falso negativo — ataque real não detectado	Catastrófica	Ocasional	Crítico
HZ-02	Falso positivo — cliente legítimo acusado	Crítica	Provável	Crítico
HZ-03	Estouro de memória em log grande	Crítica	Provável	Crítico

***História breve do HZ-02.** Detector marca o crawler do Google como força bruta. Sysadmin bloqueia o IP. SEO da empresa cai por 48 horas. Mitigação: threshold configurável, allowlist de User-Agents, contexto do match no relatório.*

Modelo de ameaças — STRIDE e ataque AT-01

Categoria	Onde ocorre no Log Sentinel	Defesa principal
Spoofing	Substituição do binário PyInstaller publicado	Checksum em release notes
Tampering	Troca do log entre leitura e hash (TOCTOU)	Hash em streaming junto da leitura
Repudiation	Operador nega ter rodado a análise	Versão + parâmetros + timestamp (S7)
Information Disclosure	JSON com <code></div></code> aberto no browser	B5 — sanitização da saída
Denial of Service	Log envenenado / ReDoS no parser	B4 + regex sem backtracking
Elevation of Privilege	Dependência transitiva maliciosa	Dependabot + permissions: mínimas

AT-01 Log Poisoning

1. Atacante envia requisição com User-Agent malicioso.
2. Linha é gravada no log Apache.
3. Operador abre relatório JSON em visualizador web.
4. Script executa na máquina do operador.

Contramedida: B5 — escape de `<`, `>` e aspas na serialização JSON.

Acompanhamento — commits, marcos, previsões

Commits por integrante (abr–jun 2026)

Integrante	Papel	Commits
Elder	CI/CD, releases, docs	9
Aryan	Core (parsers, modelos)	9
Rodrigo	CLI (Typer)	4
Helena	GUI (PySide6)	2

Total: 24 commits.

Marcos em 16/06/2026

Marco	Data	Status
AV1 Ambiente	15/04	Ativo (100%)
AV2 Testes	13/05	Ativo (100%)
AV3 Documentos	17/06	Ativo (100%)
AV3 Demonstração	17/06	Parcial (30%)
Sprint 1 Core MVP	13/05 → 08/06	Parcial (15%)
AV4 Seminário	22/07	Planejado
AV5 Riscos e Qualidade	12/08	Planejado

Reconhecemos lacuna entre 13/05 e 16/06. Plano de recuperação no GANTT atualizado.

Demonstração e próximos passos

Demonstrável hoje:

- Teste verde: `test_armazena_campos`.
- CLI base: `analyze` e `batch`.
- GUI `MainWindow` em `PySide6`.
- CI verde em master.
- Cinco docs em `docs/av3/`.

Mockup de interface — a inserir

Roadmap até AV5:

Janela	Entrega
16/06 – 30/06	Parser + BruteForce + DAO + CLI + GUI
01/07 – 15/07	Release v0.1.0 (Linux + Windows)
22/07	AV4 — Seminário
23/07 – 12/08	Hardening B4/B5/B8 + S1/S5/S6; cob. 80%
12/08	AV5 — Riscos e Qualidade

Obrigado.

Perguntas?

`github.com/202316360036/log-sentinel`

Branch: `docs/av3-especificacao`

Documentos AV3: `docs/av3/`

Referências

1. Reason, J. (1990). *Human Error*. Cambridge University Press.
2. Avizienis, A.; Laprie, J.-C.; Randell, B.; Landwehr, C. (2004). *Basic Concepts and Taxonomy of Dependable and Secure Computing*. IEEE TDSC, 1(1), 11–33.
3. Leveson, N. (2011). *Engineering a Safer World*. MIT Press.
4. Microsoft (1999). *STRIDE Threat Modeling*.
5. NIST SP 800-30 Rev. 1. *Guide for Conducting Risk Assessments*.
6. OWASP Top 10 (2021).
7. ISO/IEC 25010:2011 — *Software Quality Requirements and Evaluation*.
8. LGPD — Lei 13.709/2018.